

Preprocessing Steps Documentation

1. Data Cleaning:

- **Handling Missing Values:**
 - Missing values were addressed through imputation or exclusion where appropriate.
- **Removing Duplicates:**
 - Duplicate rows were identified and removed to ensure data integrity.

2. Feature Engineering Steps:

- **Datetime Conversion:**
 - **Columns:** `datetime_s` and `datetime_e`
 - **Description:** Converted datetime columns to datetime objects using `pd.to_datetime`.
 - **Purpose:** Enables time-based analysis and modeling.
- **Categorical Encoding:**
 - **Columns:** `event_cat`, `event_sev`, `streetname`, `light_condition`, `accident_sev`
 - **Description:** Encoded categorical variables using `LabelEncoder`.
 - **Purpose:** Transformed categorical data into numerical format suitable for machine learning models.
- **Datetime Conversion Details:**
 - **Implementation:** Used `pd.to_datetime` function.
 - **Code:**

```
python
Copy code
for column in ['datetime_s', 'datetime_e']:
    df[column] = pd.to_datetime(df[column])
```
 - **Benefits:**
 - Facilitates chronological analysis of events.
 - Supports time-based feature extraction (e.g., day of week, hour of day).
- **Categorical Encoding Details:**
 - **Implementation:** Used `LabelEncoder` for each categorical column.
 - **Code:**

```
python
Copy code
encoder = LabelEncoder()
for column in ['event_cat', 'event_sev', 'streetname',
               'light_condition', 'accident_sev']:
    df[column] = encoder.fit_transform(df[column])
if column == 'streetname':
```

```
np.save('weights_files/classes_streetname.npy',
encoder.classes_)
```

- **Benefits:**

- Converts textual categories into numeric representations.
- Prepares data for input into machine learning models that require numerical inputs.

- **File Saving (Optional):**

- **Details:** Saved encoded classes for `streetname` using `np.save`.
- **Purpose:** Preserves the mapping of categorical labels to numerical codes for future reference or deployment.

- **Integration with Additional Feature Engineering (Not Shown):**

- **Details:** Calls `feature_eng.main(data)` after initial preprocessing.
- **Purpose:** Executes additional feature transformations specific to the application domain (e.g., domain-specific calculations, derived features).

3. Outlier Handling:

- **Handling Outliers in `accident_prob`:**

- Rows with `accident_prob` exceeding 100 were removed to prevent bias in predictions, as accident probability is a percentage and cannot exceed 100.

4. Integration and Data Retrieval:

- **Data Retrieval:**

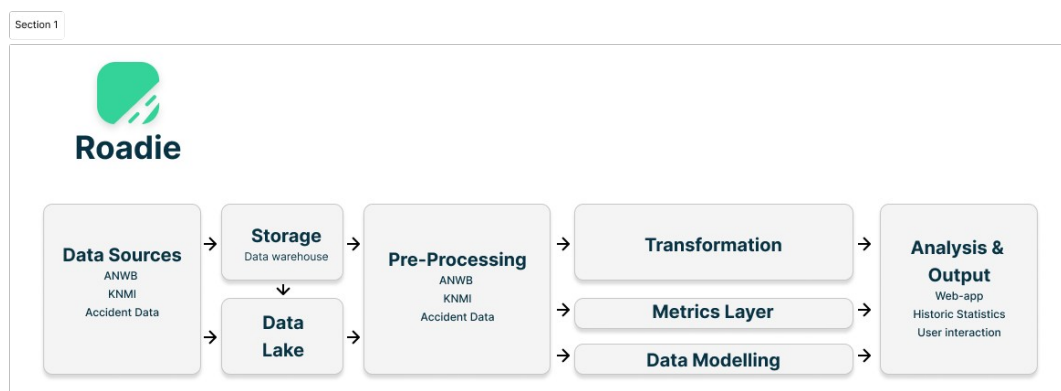
- Data was fetched from the PostgreSQL database table `group14_warehouse.regression_data` using a SQL query.

5. Data Normalization:

- **Normalization:**

- Data normalization was performed using `StandardScaler` to ensure consistent scale across features.

6. Schema Diagram of Data Warehouse:



-